

Space invaders

Jhon Pulgarin, Andrés Eusse, Yesid Hurtado

Abril 2026

1 Descripción general del proyecto

El proyecto implementa una versión simplificada de *Space Invaders* en Jack (Nand2Tetris), orientada a ejecutarse en **VMEulator** con uso moderado de memoria. El jugador controla una nave tipo cohete que se mueve horizontalmente y dispara (en etapas posteriores). Los enemigos son estáticos (no disparan) y no existen barreras defensivas, acorde con las restricciones definidas.

2 Funcionamiento general

El flujo principal es:

- Se dibuja el escenario estático (marco, título, enemigos y luna).
- Se instancia la nave y se dibuja en pantalla.
- En un ciclo de juego se lee el teclado para mover la nave con flechas izquierda/derecha.
- Para optimizar rendimiento, no se redibuja todo el tablero: solo se borra y se vuelve a dibujar el cohete en su nueva posición.

El movimiento usa saltos de 16 px para mantener la alineación con la memoria de pantalla (cada palabra representa 16 bits).

3 Estructura del proyecto

- **Main.jack**: punto de entrada. Crea una instancia de **Game** y ejecuta el ciclo principal.
- **Game.jack**: controla el ciclo del juego, lectura de teclado y actualización del movimiento.
- **Scene.jack**: dibuja los elementos estáticos (marco, enemigos, luna, título).
- **Rocket.jack**: define la nave como objeto, con métodos para dibujar, borrar y moverse.

4 Descripción de cada archivo .jack

Main.jack

Responsable de iniciar el programa. Su función principal crea el objeto `Game` y llama a `run()`.

Game.jack

Contiene el ciclo del juego. Lee el teclado con `Keyboard.keyPressed()` y, según la tecla, llama a `moveLeft` o `moveRight` del cohete. También regula la velocidad con `Sys.wait`.

Scene.jack

Dibuja una sola vez los elementos estáticos del escenario. Los enemigos se generan con bucles para formar filas y columnas. La luna y otros elementos usan escritura directa en memoria mediante `Memory.poke` para optimizar el sprite.

Rocket.jack

Encapsula la nave como un objeto. Usa un método `blit` que escribe o borra el sprite llamando internamente a `Memory.poke`. Esto evita duplicar el listado de valores para dibujar y para borrar.

5 Dificultades encontradas

- **Alineación en memoria:** los sprites basados en `Memory.poke` requieren que la coordenada `x` sea múltiplo de 16 para evitar desalineaciones.
- **Tamaño de sprites:** redimensionar manualmente un bitmap obliga a regenerar los valores de memoria.
- **Rendimiento:** redibujar toda la escena en cada frame era costoso; se resolvió actualizando solo el cohete.
- **OOP en Jack:** se reorganizó el código en clases, evitando el uso de `this` en llamadas internas, que no es valido en Jack.

6 Link Github con código y vídeo de sustentación

<https://github.com/usuario/repositorio>

7 Muestra Gráfica

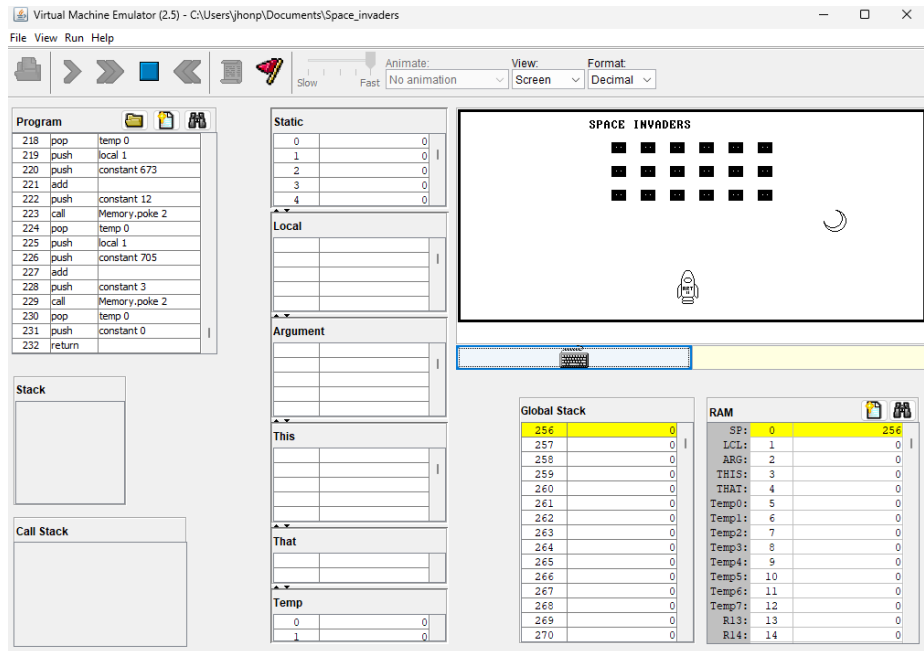


Figure 1: Pantalla principal del juego en VM

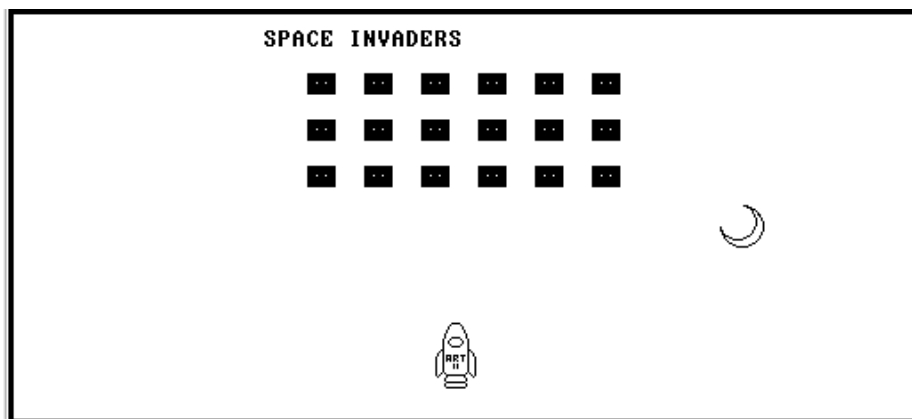


Figure 2: Zoom en detalle de la pantalla